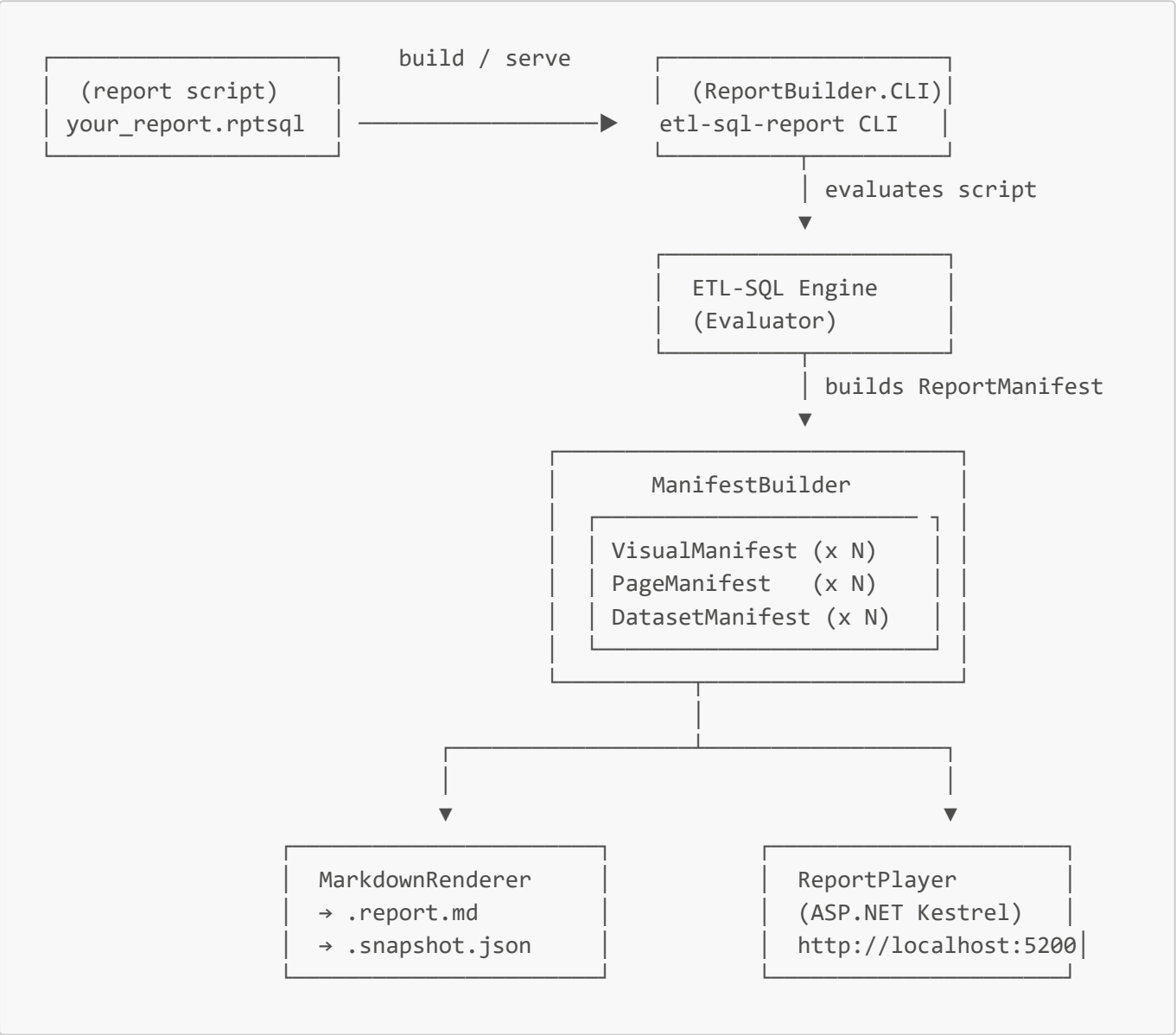
Report-SQL Scripting Guide

Report-SQL extends ETL-SQL with dedicated statement types for building interactive dashboards: `SET REPORT TITLE`, `CREATE DATASET`, `CREATE VISUAL`, `CREATE PAGE`, `CREATE CONTAINER`, and `CREATE NAVIGATION` — plus a CLI build tool and a live web dashboard for serving reports.

How it works — architecture overview



A `.rptsql` file is a normal ETL-SQL script that may also contain Report-SQL statements. The engine evaluates it exactly like any `.etlsql` file; the new statements register definitions in the execution context. After evaluation the `ManifestBuilder` snapshots the data and produces a `ReportManifest` — a serializable JSON structure consumed by both the static Markdown renderer and the live web dashboard.

Data Sources for Visuals

The recommended way to supply data to a visual is a named temp table populated with a `SELECT INTO` or inline `UNION ALL`:

```
SELECT 'Product A' AS Label, 100 AS Value INTO #kpi
UNION ALL
SELECT 'Product B', 250;

CREATE VISUAL KpiCard AS CARD (SOURCE = #kpi, MAPPINGS(VALUE = Value, LABEL =
Label));
```

Temp tables are reusable across multiple visuals, debuggable with a plain `SELECT`, and consistent with the rest of ETL-SQL.

Quick start

```
-- 1. Connect and pull data
CREATE CONNECTION c ON FLATFILE('data/sales.csv');

SELECT region, SUM(revenue) AS revenue
INTO #summary
FROM c
GROUP BY region;

-- 2. Define a visual
CREATE VISUAL SalesChart AS BAR (
  SOURCE = #summary,
  MAPPINGS (X = region, Y = revenue),
  OPTIONS (
    X_AXIS (LABEL = 'Region'),
    Y_AXIS (LABEL = 'Revenue ($)')
  )
);

-- 3. Arrange on a page (STRUCTURE uses CSS grid-template-areas)
CREATE PAGE Main AS LAYOUT (
  STRUCTURE = 'A',
  MAP ('A' = SalesChart)
);
```

Save as `report.rptsql`, then:

```
etl-sql-report build report.rptsql        # → report.report.md +
report.snapshot.json
etl-sql-report serve report.rptsql        # → opens http://localhost:5200
```

SET REPORT TITLE / SET REPORT DESCRIPTION

Sets the report title and description displayed in the dashboard header and catalog page.

```
SET REPORT TITLE = 'Sales Dashboard';
SET REPORT DESCRIPTION = 'Regional and product-level revenue analysis for Q1
2026.';
```

Both statements are optional. If omitted the script filename is used as the title.

CREATE VISUAL

```
CREATE [OR ALTER] VISUAL <name> AS <TYPE> (
  [SOURCE = <source>,]
  [TITLE = '<string>',]
  [SUBTITLE = '<string>',]
  [TOOLTIP = '<string>',]
  [MAPPINGS (role = column, ...),]
  [OPTIONS (key = value, ..., X_AXIS (...), Y_AXIS (...), COLORS (...), LEGEND
(...)),]
  [STYLE = <styleName> | STYLE (key = value, ...),]
  [SERIES (type column, ...),]
  [ACTIONS (trigger = action, ...)]
);
```

All clauses inside the outer `( )` are separated by commas. The closing `)` ends the statement. `SOURCE` is required for all types except `TEXT`, `DATEPICKER`, `SLIDER`, and `SEARCH`.

Visual types

Type	Description	Renderer
`BAR`	Vertical bar chart. Supports grouping via a `SERIES` mapping.	ECharts
`HBAR`	Horizontal bar chart. Same mappings as BAR, bars run left-to-right.	ECharts
`LINE`	Line chart. Supports multiple series and smooth curves.	ECharts
`SCATTER`	X/Y scatter plot. Each row becomes one point.	ECharts
`PIE`	Pie chart. One slice per row.	ECharts
`DONUT`	Donut chart. Same mappings as PIE; center hole rendered.	ECharts
`COMBO`	Combined bar + line chart. Use `SERIES (BAR col, LINE col)` to assign series types.	ECharts
`BOXPLOT`	Box-and-whisker plot for distribution visualization.	ECharts
`TREEMAP`	Hierarchical area chart. One rectangle per row, sized by value.	ECharts

Type	Description	Renderer
HEATMAP	Grid heatmap. Requires X, Y, and VALUE mappings.	ECharts
GAUGE	Radial KPI gauge. Single VALUE from first data row against a min/max arc.	ECharts
FUNNEL	Conversion funnel. Each row is one stage (LABEL + VALUE).	ECharts
WATERFALL	Cumulative change chart. Positive values rise, negative values fall.	ECharts
TABLE	Paginated, scrollable data grid. Supports FORMATTING for conditional cell colors.	HTML <table>
CARD	Single large KPI number with an optional label.	Styled
TEXT	Free-form text or HTML block. Uses the DEFAULT clause, not a SOURCE query.	
SLICER	Dropdown parameter selector. SOURCE provides the option list.	<select>
DATEPICKER	Date input control. No SOURCE required.	<input type="date">
SLIDER	Numeric range slider. No SOURCE required.	<input type="range">
MULTISELECT	Multi-value checkbox list. SOURCE provides the option list.	Checkbox list
SEARCH	Free-text search box with debounce. No SOURCE required.	<input type="text">

SOURCE

The SOURCE clause provides the data for the visual. It can be:

```
-- a) A pre-populated temp table (set earlier in the same script)
SOURCE = #my_temp_table

-- b) An inline SELECT (must be wrapped in parentheses)
SOURCE = (SELECT region, SUM(revenue) AS rev FROM #summary GROUP BY region)
```

Inline SELECTs are evaluated at build time and their results are snapshotted into the manifest. If you need the visual to refresh independently, use CREATE DATASET and reference the dataset by name.

TEXT, DATEPICKER, SLIDER, and SEARCH visuals do not require SOURCE. MULTISELECT requires a SOURCE to populate the option list.

TITLE and SUBTITLE

```
CREATE VISUAL RevenueCard AS CARD (
  SOURCE = ...,
  TITLE    = 'Total Revenue',
  SUBTITLE = 'All regions, YTD',
  MAPPINGS (VALUE = revenue)
);
```

`TITLE` overrides the visual name as the chart heading. `SUBTITLE` appears below the title in smaller text.

TOOLTIP

Adds hover-triggered content shown when the user hovers over the visual's title area. Three forms are supported:

1. Plain text

```
TOOLTIP = 'Sum of all regional revenue, YTD.'
```

2. Named container reference

References an existing `CREATE CONTAINER` by name. The container's visuals are rendered as a popover panel. The hovered value is injected as a parameter (`@hover_value`) so tooltip visuals can filter to the point being inspected.

```
CREATE VISUAL RegionSparkline AS LINE (
  SOURCE = (SELECT month, SUM(revenue) AS revenue FROM #summary
            WHERE region = @hover_value GROUP BY month),
  MAPPINGS (X = month, Y = revenue)
);

CREATE CONTAINER RegionTooltip AS BOX (
  VISUALS (RegionSparkline)
);

CREATE VISUAL RevenueByRegion AS BAR (
  SOURCE   = (SELECT region, SUM(revenue) AS revenue FROM #summary GROUP BY
region),
  MAPPINGS (X = region, Y = revenue),
  TOOLTIP  = RegionTooltip       -- container reference
);
```

3. Inline anonymous container

Defines tooltip content inline — an optional markdown string followed by a `VISUALS` list. The outer `( )` delimit the inline block; markdown and `VISUALS` are separated by a comma.

```
TOOLTIP = ('**Revenue trend** for the hovered region', VISUALS(RegionSparkline))

-- markdown only (no chart)
TOOLTIP = ('Click a bar to drill down by month.')

-- visuals only (no markdown)
TOOLTIP = (VISUALS(RegionSparkline, RegionTable))
```

Full example:

```
CREATE VISUAL RevenueCard AS CARD (
  SOURCE   = #kpis,
  TITLE    = 'Total Revenue',
  TOOLTIP  = ('**Total Revenue** — sum of all regional revenue, YTD.',
VISUALS(TrendSparkline)),
  MAPPINGS (VALUE = revenue, LABEL = label)
);
```

`@hover_value` is set to the X-axis or LABEL value of the hovered data point. Tooltip visuals that reference `@hover_value` in their inline `SELECT` are re-queried on each hover. `TOOLTIP` applies to all visual types, `CREATE PAGE`, `CREATE CONTAINER`, and `CREATE BUTTON`.

MAPPINGS

Maps the columns returned by `SOURCE` to semantic **roles** expected by the renderer. Roles are case-insensitive.

BAR, HBAR, and LINE

Role	Description
`X`	Category axis column (string or date). Required.
`Y`	Value axis column (numeric). Required.
`SERIES`	Optional grouping column. Each distinct value becomes a separate coloured series.

```
-- Simple: single series
MAPPINGS (X = month, Y = revenue)

-- Grouped: one line/bar per region
MAPPINGS (X = month, Y = revenue, SERIES = region)
```

SCATTER

Role	Description
X	Horizontal axis column (numeric).
Y	Vertical axis column (numeric).

```
MAPPINGS (X = score, Y = rank)
```

PIE and DONUT

Role	Description
LABEL	Column to use as the slice label.
VALUE	Column to use as the slice size (numeric).

```
MAPPINGS (LABEL = category, VALUE = total)
```

CARD

Role	Description
LABEL	Small caption rendered above the number. If omitted the column name is used.
VALUE	The primary number or string to display large.

```
MAPPINGS (VALUE = val, LABEL = lbl)
```

TEXT

TEXT visuals render free-form string content. No SOURCE is required; the content is provided via the DEFAULT clause.

```
CREATE VISUAL WelcomeText AS TEXT (
  TITLE   = 'Welcome',
  DEFAULT = '### Hello, World!\nThis is a *markdown-enabled* text block.'
);
```

SLICER

SLICER uses SOURCE to provide option rows and ACTIONS to bind the selection to a parameter. The MAPPINGS clause specifies which column from the source holds the display value.

```
CREATE VISUAL RegionFilter AS SLICER (
  SOURCE  = (SELECT DISTINCT region FROM #summary ORDER BY region),
  MAPPINGS (VALUE = region),
  ACTIONS  (ON_CHANGE = SET_PARAMETER(@region, region)),
  DEFAULT  = 'All'
);
```

The `DEFAULT` option on a SLICER pre-selects that value in the dropdown on load. It is cosmetic only — it does not declare the page parameter. The corresponding `WITH PARAMETERS (@region = 'All')` on the `CREATE PAGE` statement is what makes `@region` available to visual queries from the first render.

MULTISELECT

Same pattern as SLICER: `SOURCE` provides options, `ACTIONS` binds the selection.

```
CREATE VISUAL CategoryFilter AS MULTISELECT (
  SOURCE = (SELECT DISTINCT category FROM #products ORDER BY category),
  MAPPINGS (VALUE = category),
  ACTIONS (ON_CHANGE = SET_PARAMETER(@category, category))
);
```

SLIDER

`SLIDER` is a numeric range control. No `SOURCE` is required. Set bounds and step in `OPTIONS`, then bind the value to a page parameter via `ACTIONS`.

```
CREATE VISUAL YearSlider AS SLIDER (
  TITLE   = 'Year',
  TOOLTIP = 'Drag to filter by year',
  OPTIONS (MIN = 2020, MAX = 2026, STEP = 1, DEFAULT = 2024),
  ACTIONS (ON_CHANGE = SET_PARAMETER(@year, value))
);
```

Option key	Description
`MIN`	Minimum slider value (numeric). Default `0`.
`MAX`	Maximum slider value (numeric). Default `100`.
`STEP`	Increment between positions. Default `1`.
`DEFAULT`	Initial value when the page loads.

DATEPICKER

`DATEPICKER` is a date input control. No `SOURCE` is required.

```
CREATE VISUAL StartDate AS DATEPICKER (
  TITLE   = 'Start Date',
  TOOLTIP = 'Filter results from this date',
  OPTIONS (MIN = '2020-01-01', MAX = '2026-12-31', DEFAULT = '2024-01-01'),
  ACTIONS (ON_CHANGE = SET_PARAMETER(@startDate, value))
);
```

Option key	Description
MIN	Earliest selectable date ('YYYY-MM-DD').
MAX	Latest selectable date ('YYYY-MM-DD').
DEFAULT	Initial date when the page loads.

SEARCH

SEARCH is a free-text input box with debounce. No SOURCE is required.

```
CREATE VISUAL ProductSearch AS SEARCH (
  TITLE   = 'Search',
  OPTIONS (PLACEHOLDER = 'Type a product name...', DEFAULT = ''),
  ACTIONS (ON_CHANGE = SET_PARAMETER(@searchTerm, value))
);
```

Option key	Description
PLACEHOLDER	Ghost text shown when the box is empty.
DEFAULT	Initial text value when the page loads.

HEATMAP

Role	Description
X	Horizontal axis column.
Y	Vertical axis column.
VALUE	Cell intensity value (numeric).

```
MAPPINGS (X = hour, Y = weekday, VALUE = count)
```

TREEMAP

Role	Description
LABEL	Rectangle label column.
VALUE	Rectangle size column (numeric).

```
MAPPINGS (LABEL = product, VALUE = revenue)
```

GAUGE

Role	Description
VALUE	The current gauge reading (numeric, first data row only).
MAX	Optional column providing the maximum value for the arc.
LABEL	Optional label shown at the center of the gauge.

```
CREATE VISUAL RevenueKpi AS GAUGE (
  SOURCE = (SELECT 73 AS pct, 100 AS target, 'Revenue Target' AS lbl),
  MAPPINGS (VALUE = pct, MAX = target, LABEL = lbl),
  OPTIONS (MIN = 0, MAX = 100, TITLE = 'Revenue vs Target')
);
```

`MIN` and `MAX` options override column-derived bounds. Both default to `0` / `100` when omitted.

FUNNEL

Role	Description
LABEL	Stage name column.
VALUE	Numeric value for each stage (determines bar width).

```
CREATE VISUAL SalesFunnel AS FUNNEL (
  SOURCE = (SELECT Stage, Leads FROM #funnel ORDER BY Leads DESC),
  MAPPINGS (LABEL = Stage, VALUE = Leads)
);
```

WATERFALL

Role	Description
X	Category / period column.
Y	Numeric delta — positive values rise, negative values fall.

```
CREATE VISUAL CashFlow AS WATERFALL (
  SOURCE = (SELECT Period, Delta FROM #cashflow),
  MAPPINGS (X = Period, Y = Delta)
);
```

Use the `COLORS (positive = '#5cb85c', negative = '#d9534f')` option inside `OPTIONS (COLORS (...))` to customise bar colors.

BOXPLOT

Role	Description
`X`	Category axis column (string).
`Q1`	First quartile (25th percentile) value.
`MEDIAN`	Median (50th percentile) value.
`Q3`	Third quartile (75th percentile) value.
`LOW`	Lower whisker value (e.g. min or 1.5·IQR bound).
`HIGH`	Upper whisker value (e.g. max or 1.5·IQR bound).

```
CREATE VISUAL PriceDistribution AS BOXPLOT (
  SOURCE = (SELECT category, low, q1, median, q3, high FROM #price_stats),
  MAPPINGS (X = category, LOW = low, Q1 = q1, MEDIAN = median, Q3 = q3, HIGH =
high),
  TITLE = 'Price Distribution by Category'
);
```

TABLE

TABLE visuals use all columns returned by `SOURCE` in definition order. No `MAPPINGS` clause is needed.

See Conditional Formatting for applying cell colors to TABLE visuals.

COMBO

COMBO visuals use the `SERIES` block to assign which columns render as bars vs lines. The `X` mapping provides the category axis.

```
CREATE VISUAL SalesCombo AS COMBO (
  SOURCE = (SELECT month, revenue, units FROM #summary),
  MAPPINGS (X = month),
  SERIES (BAR revenue, LINE units)
);
```

OPTIONS

General key/value options plus optional axis sub-blocks. All are optional:

```
OPTIONS (
  -- flat options:
  TITLE           = 'Revenue Over Time',
  STACKED         = ON,
  SMOOTH          = ON,
  FORMAT          = 'N0',
  LEGEND_POSITION = BOTTOM,   -- TOP | BOTTOM | LEFT | RIGHT

  -- axis sub-blocks (BAR, HBAR, LINE, SCATTER only):
  X_AXIS (
    LABEL = 'Month',
    MIN   = 0,
    MAX   = 100
  ),
  Y_AXIS (
    LABEL = 'Revenue ($)',
    MIN   = 0
  ),

  -- color map (any chart type):
  COLORS (
    'North' = '#4e79a7',
    'South' = '#f28e2b'
  )
)
```

Flat OPTIONS reference

Key	Applies to	Values	Description
`TITLE`	All chart types	Any string	Chart title. Defaults to the visual name. Prefer the top-level `TITLE` clause.
`STACKED`	BAR, HBAR, LINE	`ON` / `OFF`	Stack series on top of each other. Default `OFF`.
`SMOOTH`	LINE	`ON` / `OFF`	Smooth curves via bezier interpolation. Default `OFF`.
`FORMAT`	CARD	.NET format string	Applies a numeric format to the VALUE column (e.g. `N0`, `C2`, `P1`).
`LEGEND_POSITION`	All chart types	`TOP` / `BOTTOM` / `LEFT` / `RIGHT`	Legend placement. Default `BOTTOM`.

X_AXIS / Y_AXIS sub-block options

Key	Values	Description
LABEL	Any string	Human-readable axis label.
MIN	Numeric	Force axis minimum.
MAX	Numeric	Force axis maximum.

COLORS

Maps category values to specific hex colors. Key is the category value (quoted if it contains spaces); value is a CSS color string.

```
COLORS (
  'East'  = '#4e79a7',
  'West'  = '#f28e2b',
  'North' = '#76b7b2'
)
```

LEGEND_POSITION

Controls legend placement. Use as a flat key in the OPTIONS block:

```
OPTIONS (LEGEND_POSITION = TOP)     -- TOP | BOTTOM | LEFT | RIGHT
```

FORMATTING (Conditional Cell Colors) {#conditional-formatting}

Applies CSS colors to TABLE visual cells based on column value comparisons. Each rule specifies a column, a comparison operator, a threshold, and a color:

```
CREATE VISUAL FinancialSummary AS TABLE (
  SOURCE = (SELECT Category, Revenue, Margin FROM #summary),
  FORMATTING (
    Revenue < 0       THEN 'red',
    Revenue >= 100000 THEN '#28a745',
    Margin < 0.05     THEN 'orange'
  )
);
```

Rule syntax: `column operator threshold THEN 'color'`

Operator	Meaning
<	Less than
>	Greater than

Operator	Meaning
`<=`	Less than or equal
`>=`	Greater than or equal
`=`	Equal
`<>`	Not equal

Multiple rules for the same column are evaluated top-to-bottom; the first matching rule wins. Numeric thresholds use numeric comparison; string thresholds use string equality (`=` / `<>`). `color` may be any CSS color value (`'red'`, `'#ff0000'`, `'rgba(255,0,0,0.5)'`).

OVERLAYS

Adds reference lines and statistical curves on top of BAR, LINE, HBAR, and SCATTER visuals. Each entry specifies an overlay type, a line style, and optional color and label.

```
OVERLAYS (
  <type>  AS SOLID|DASHED|DOTTED [WITH (COLOR = '<css>', LABEL = '<text>')],
  ...
)
```

Overlay types

Type	Description	Parameter
`GOAL(n)`	Horizontal line at a fixed value	`n` — the target value (required)
`AVERAGE`	Horizontal line at the computed mean of the Y column	None
`MOVING_AVG(n)`	Rolling average line smoothed over `n` periods	`n` — window size (required)
`LINEAR`	Straight line fitted by linear regression (least squares)	None
`EXPONENTIAL`	Exponential curve fit (`y = ae^(bx)`)	None
`LOGARITHMIC`	Logarithmic curve fit (`y = a + b·ln(x)`)	None
`POWER`	Power curve fit (`y = a·x^b`)	None
`POLYNOMIAL(n)`	Polynomial curve of degree `n` fitted by least squares	`n` — degree (required)

`GOAL` and `AVERAGE` render as ECharts `markLine` overlays on the chart — they are always horizontal and do not require additional data points. `MOVING_AVG`, `LINEAR`, and the regression types render as additional line series computed at build time from the Y column values.

Line styles

Style	Description
`SOLID`	Continuous line
`DASHED`	Evenly dashed line
`DOTTED`	Dotted line

WITH clause

Both `COLOR` and `LABEL` are optional:

- `COLOR` — any CSS color string (`'#e74c3c'`, `'rgb(0,0,0)'`). Defaults to `#888888`.
- `LABEL` — text shown on the overlay in the chart legend. Defaults to the overlay type name.

Examples

```
-- Sales line chart with goal, average, and 3-month moving average
CREATE VISUAL RevenueByMonth AS LINE (
  SOURCE   = (SELECT month, SUM(revenue) AS revenue FROM #summary GROUP BY month),
  MAPPINGS (X = month, Y = revenue),
  OVERLAYS (
    GOAL(100000)  AS DASHED WITH (COLOR = '#e74c3c', LABEL = 'Annual Target'),
    GOAL(80000)   AS DOTTED WITH (COLOR = '#e67e22', LABEL = 'Minimum'),
    AVERAGE       AS DASHED WITH (COLOR = '#3498db', LABEL = 'Mean'),
    MOVING_AVG(3) AS SOLID  WITH (COLOR = '#2ecc71', LABEL = '3-Month Avg')
  )
);

-- Scatter plot with linear regression and polynomial fit
CREATE VISUAL ScoreVsRank AS SCATTER (
  SOURCE   = (SELECT score, rank FROM #results),
  MAPPINGS (X = score, Y = rank),
  OVERLAYS (
    LINEAR      AS DASHED WITH (COLOR = '#9b59b6', LABEL = 'Linear Fit'),
    POLYNOMIAL(2) AS DOTTED WITH (COLOR = '#e67e22', LABEL = 'Poly Fit')
  )
);
```

Multiple `GOAL` lines are supported — just add additional `GOAL(n)` entries. `OVERLAYS` applies to BAR, HBAR, LINE, and SCATTER visuals; it is ignored on PIE, DONUT, TABLE, CARD, and filter controls.

CROSS_FILTER

Setting `CROSS_FILTER = ON` in the `OPTIONS` clause of a chart visual makes it act as a cross-filter source. Clicking a data point broadcasts a filter to all TABLE visuals on the same page that also have `CROSS_FILTER = ON`. Clicking the same value again clears the filter.

```
CREATE VISUAL SalesByRegion AS BAR (
  SOURCE = (SELECT Region, Revenue FROM #sales),
  MAPPINGS (X = Region, Y = Revenue),
  OPTIONS (CROSS_FILTER = ON)
);

CREATE VISUAL SalesDetail AS TABLE (
  SOURCE = (SELECT Region, Product, Revenue FROM #sales),
  OPTIONS (CROSS_FILTER = ON)  -- becomes a filter target
);
```

When the user clicks "West" in the bar chart, the TABLE is filtered to show only rows where `Region = 'West'`. Cross-filtering operates client-side using the data already in the manifest — no server round-trip is needed.

STYLE

Applies visual-level styling properties. Visual-level STYLE takes precedence over page-level THEME.

```
STYLE (
  THEME      = dark,          -- dark | light
  HEIGHT     = 400,           -- pixels
  WIDTH      = 600,           -- pixels
  BACKGROUND = '#1a1a2e',
  BORDER     = '1px solid #333'
)
```

STYLE property reference

Key	Example	Applies to	Description
`THEME`	`dark`	Visual, Page	ECharts theme (`dark` or `light`).
`BACKGROUND-COLOR`	`'#1a1a2e'`	Any	Background color of the card/page.
`COLOR`	`'#ffffff'`	Any	Default text color.
`BORDER`	`'1px solid #444'`	Any	CSS border definition.
`BORDER-RADIUS`	`'8px'`	Any	Corner rounding.
`FONT-SIZE`	`'14px'`	Any	Base font size for textual content.
`PADDING`	`'12px'`	Any	Inner spacing.
`HEIGHT`	`200`	Container	Fixed height for SCROLL containers.

Key	Example	Applies to	Description
WIDTH	'100%'	Any	Visual width (e.g., '100%', '400px').
TOOLTIP	'Hover text'	Visual	Floating help text. Prefer the top-level TOOLTIP clause; this key is accepted here for backwards compatibility.
Z-INDEX	100	Any	Layer stacking order.
SHADOW	ON / OFF	Visual	Enable/disable visual card shadow.

```
### ACTIONS

Actions wire up interactive behavior in the live dashboard:

```sql
ACTIONS (
  ON_CLICK  = DRILL_DOWN(Target = DetailChart, Key = region),
  ON_CHANGE = SET_PARAMETER(@region, region)
)
```

Trigger	Description
ON_CLICK	Fires when the user clicks a chart element (bar, slice, point).
ON_CHANGE	Fires when a SLICER, MULTISELECT, DATEPICKER, SLIDER, or SEARCH value changes.

DRILL_DOWN

```
ON_CLICK = DRILL_DOWN(Target = <VisualName>, Key = <column>)
```

When clicked, passes the selected row's value in Key as a filter into the target visual's inline SELECT. The target visual is re-queried with the key value injected into the parameter context.

SET_PARAMETER

```
ON_CHANGE = SET_PARAMETER(@paramName, <columnRef>)
```

Sets the named @param to the selected value. Any visual whose inline SELECT references @paramName is automatically re-queried.

CREATE DATASET
```

Pre-computes a named temp table that can be independently refreshed and optionally encrypted or compressed. Use this when multiple visuals share the same expensive base query, or when you want separate refresh cadences.

```
CREATE DATASET #<name>
  [REFRESH EVERY '<interval>']
  [TTL = '<duration>']
  [COMPRESS = ON|OFF]
  [ENCRYPT = MACHINE | PASSWORD | KEYFILE]
  [PASSWORD = '<password>']
  [KEYFILE  = '<path>']
AS ( SELECT ... );
```

Encryption modes

Mode	Description
`ENCRYPT = MACHINE`	Encrypts using a machine-bound key (DPAPI on Windows; OS keyring on Linux/macOS). No password or key file needed. Snapshot can only be decrypted on the same machine.
`ENCRYPT = PASSWORD, PASSWORD = '...'`	AES encryption with a user-supplied password. Portable — can be decrypted on any machine with the password.
`ENCRYPT = KEYFILE, KEYFILE = '...'`	AES encryption using a key file at the specified path. Portable with the key file.

Full example

```
-- Machine-bound (simplest — no credentials to manage)
CREATE DATASET #sales_snap
  REFRESH EVERY '1h'
  TTL = '24h'
  COMPRESS = ON
  ENCRYPT = MACHINE
  AS (SELECT region, product, SUM(revenue) AS revenue
      FROM sales
      GROUP BY region, product);

-- Password-protected (portable)
CREATE DATASET #sales_secure
  ENCRYPT = PASSWORD
  PASSWORD = 'MyS3cretPhrase'
  AS (SELECT * FROM sensitive_table);

-- Key-file protected
CREATE DATASET #sales_keyfile
  ENCRYPT = KEYFILE
```

```
    KEYFILE = 'C:\keys\report.key'
    AS (SELECT * FROM sales);
```

CREATE DATASET clause reference

Clause	Required	Description
`#<name>`	Yes	Temp table name. The `#` prefix is automatically added if omitted.
`REFRESH EVERY '<interval>'`	No	Re-compute interval. Format: `<n>s`, `<n>m`, `<n>h`, or `<n>d` (e.g. `'30m'`, `'1h'`, `'7d'`).
`TTL = '<duration>'`	No	How long a snapshot stays valid before `IsStale` returns true. Same interval format.
`COMPRESS = ON`	No	Compress the snapshot file on disk. Default `OFF`.
`ENCRYPT = MACHINE|PASSWORD|KEYFILE`	No	Encryption mode. See table above.
`PASSWORD = '<password>'`	Required when `ENCRYPT = PASSWORD`	Password for AES encryption.
`KEYFILE = '<path>'`	Required when `ENCRYPT = KEYFILE`	Absolute path to the AES key file. Linter raises an error if `ENCRYPT = KEYFILE` but `KEYFILE` is absent.
`AS ( SELECT ... )`	Yes	The query to execute. Must be the last clause before the semicolon.

CREATE PAGE

Arranges visuals and containers into a named layout. Multiple pages can be defined in one script; the web dashboard renders each as a distinct section.

```
CREATE [OR ALTER] PAGE <name> AS LAYOUT (
  [TITLE = '<string>',]
  [TOOLTIP = '<string>',]
  STRUCTURE = '<grid-template-areas>',
  MAP (
    '<slot>' = VisualOrContainerName,
    ...
  )
  [, STYLE = <styleName> | STYLE (key = value, ...)]
)
[WITH PARAMETERS (@param = default, ...)]
;
```

STRUCTURE string

`STRUCTURE` is a CSS grid-template-areas string. Slot letters appear as space-separated names within a row; rows are separated by `/`.

```
-- Single visual filling the full width
STRUCTURE = 'A'

-- Two visuals side by side
STRUCTURE = 'A B'

-- Two rows: header spanning both columns, then two below
STRUCTURE = 'A A / B C'

-- Three rows: KPI row, chart row, table row
STRUCTURE = 'A B C / D D D / E E E'
```

Each unique letter becomes a grid area. The renderer calculates column count from the maximum number of distinct letters in any single row.

MAP

`MAP` assigns each visual or container to a slot letter. Slot letters must match those used in `STRUCTURE`.

```
MAP (
  'A' = KpiCard,
  'B' = BarChart,
  'C' = LineChart,
  'D' = DataTable
)
```

STYLE on PAGE

Applies page-level styling. The `THEME` key cascades to all charts on the page unless overridden at the visual level.

```
STYLE (
  THEME      = dark,
  BACKGROUND = '#0f0f1a'
)
```

WITH PARAMETERS

Declares page-level parameters with optional default values. These drive dynamic queries when a SLICER or other filter control fires `SET_PARAMETER`.

Untyped (legacy):

```
WITH PARAMETERS (@region = 'All', @year = '2024')
```

Typed declarations — add `AS type` and the `DEFAULT` keyword:

```
WITH PARAMETERS (
    @startDate AS DATE    DEFAULT '2024-01-01',
    @endDate   AS DATE    DEFAULT '2024-12-31',
    @minSales  AS NUMBER  DEFAULT '0',
    @region    AS VARCHAR DEFAULT 'All'
)
```

Supported types: `DATE`, `DATETIME`, `NUMBER`, `INT`, `DECIMAL`, `VARCHAR`. The declared type is stored in the manifest under `parameterTypes` and can be used by the runtime for type-safe casting before passing to queries. Untyped parameters are treated as `VARCHAR`.

Defaults are applied immediately at page load — any visual whose inline SELECT references a page parameter can use it safely from the first render. The engine declares all `WITH PARAMETERS` variables with their default values when the `CREATE PAGE` statement executes, so no slicer interaction is needed for initial data to appear.

When a parameter changes the DashboardService re-evaluates all visuals whose inline SELECTs reference that parameter. Unaffected visuals are not re-queried.

Full page example

```
CREATE PAGE Overview AS LAYOUT (
  STRUCTURE = 'A B / C C / D D',
  MAP (
    'A' = TotalRevenue,
    'B' = RegionFilter,
```

```
    'C' = RevenueByRegion,
    'D' = SalesTable
  ),
  STYLE (THEME = dark)
)
WITH PARAMETERS (@region = 'All');
```

CREATE STYLE

Defines a named, reusable style that can be applied to any `CREATE VISUAL`, `CREATE PAGE`, or `CREATE CONTAINER` statement. Properties defined in the named style act as defaults; any inline `STYLE (...)` block on the target overrides them.

```
CREATE STYLE <name> (
  key = value,
  ...
);
```

Style properties are CSS-like key/value pairs (strings, numbers, or identifiers). Common keys:

Key	Example	Applies to
`BACKGROUND-COLOR`	`'#1a1a2e'`	Any
`COLOR`	`'#ffffff'`	Any
`BORDER`	`'1px solid #444'`	Any
`BORDER-RADIUS`	`'8px'`	Any
`FONT-SIZE`	`'14px'`	Any
`PADDING`	`'12px'`	Any
`HEIGHT`	`200`	Container
`WIDTH`	`'100%'`	Any
`TOOLTIP`	`'Hover text'`	Visual
`Z-INDEX`	`100`	Any
`SHADOW`	`ON`	Visual

```
-- Define shared styles once
CREATE STYLE DarkCard (
  BACKGROUND-COLOR = '#1e1e2e',
  COLOR = '#cdd6f4',
  BORDER-RADIUS = '8px',
  PADDING = '16px'
```

```
);

CREATE STYLE PanelBorder (
  border = '1px solid #444',
  border-radius = '4px'
);

-- Reference by name in CREATE VISUAL
CREATE VISUAL RevenueKpi AS CARD (
  SOURCE = #kpis,
  STYLE = DarkCard,
  MAPPINGS (VALUE = revenue, LABEL = label)
);

-- Inline overrides take precedence over the named style
CREATE VISUAL AlertKpi AS CARD (
  SOURCE = #kpis,
  STYLE = DarkCard,
  STYLE (color = '#f38ba8'),   -- override color only
  MAPPINGS (VALUE = alerts, LABEL = label)
);

-- Apply to pages and containers too
CREATE PAGE Main AS LAYOUT (
  STRUCTURE = 'A B',
  STYLE = PanelBorder,
  MAP ('A' = RevenueKpi, 'B' = AlertKpi)
);
```

> Named styles are resolved at manifest build time and merged into the target's final style map. They are not emitted as a separate entity in the manifest.

CREATE CONTAINER

Groups multiple visuals into a single layout region, optionally with scrolling. Useful when many visuals share one page slot.

```
CREATE [OR ALTER] CONTAINER <name> AS BOX|SCROLL (
  [TITLE = '<string>',]
  [SUBTITLE = '<string>',]
  [TOOLTIP = '<string>',]
  [STYLE = <styleName> | STYLE (key = value, ...),]
  VISUALS (VisualA, VisualB, ...)
);
```

Type	Description
BOX	Fixed-height container. Visuals are stacked vertically inside.

Type	Description
SCROLL	Scrollable container. Overflow visuals scroll within the container.

```
CREATE CONTAINER KpiRow AS BOX (
  STYLE (HEIGHT = 200),
  VISUALS (TotalRevenue, TotalUnits, AvgOrderValue)
);

-- Reference the container in MAP just like a visual
CREATE PAGE Main AS LAYOUT (
  STRUCTURE = 'A A / B C',
  MAP (
    'A' = KpiRow,
    'B' = RevenueChart,
    'C' = SalesTable
  )
);
```

CREATE NAVIGATION

Adds a navigation bar that controls which page is visible. The bar renders above the page content.

```
CREATE [OR ALTER] NAVIGATION <name> AS TAB|BUTTON|LINK (
  [ORIENTATION = HORIZONTAL|VERTICAL,]
  [DEFAULT = <PageName>]
)
WITH PAGES (Page1, Page2, ...);
```

Nav type	Rendering
TAB	Tab-style bar (default).
BUTTON	Pill buttons.
LINK	Separator-delimited links.

```
CREATE NAVIGATION MainNav AS TAB (
  ORIENTATION = HORIZONTAL,
  DEFAULT = Overview
)
WITH PAGES (Overview, Details, Trends);
```

If DEFAULT is omitted, the first page in the list is shown on load.

CREATE BUTTON

Adds an interactive button to a page. Buttons are placed in `MAP` slots just like visuals.

```
CREATE [OR ALTER] BUTTON <name> AS BACK|REFRESH|<customType> (
  [TITLE   = '<string>',]
  [TOOLTIP = '<string>',]
  [OPTIONS (key = value, ...),]
  [ACTIONS (trigger = action, ...),]
  [STYLE = <styleName> | STYLE (key = value, ...)]
);
```

Button types

Type	Behavior
`BACK`	Navigates to the previous page in the browser history.
`REFRESH`	Forces a full report refresh (equivalent to hitting `/api/refresh`).
`<identifier>`	Custom type — behavior driven entirely by the `ACTIONS` clause.

Examples

```
-- Navigation button
CREATE BUTTON GoBack AS BACK (
  TITLE   = '← Back',
  TOOLTIP = 'Return to the previous page'
);

-- Refresh button
CREATE BUTTON RefreshData AS REFRESH (
  TITLE   = 'Refresh',
  TOOLTIP = 'Reload all visuals from source data',
  STYLE (BACKGROUND-COLOR = '#2563eb', COLOR = '#ffffff', BORDER-RADIUS = '4px')
);

-- Custom action button
CREATE BUTTON ExportBtn AS EXPORT (
  TITLE   = 'Export CSV',
  ACTIONS (ON_CLICK = SET_PARAMETER(@export, 'csv'))
);
```

Place buttons in a page layout exactly like any visual:

```
CREATE PAGE Dashboard AS LAYOUT (
  STRUCTURE = 'A B / C C',
  MAP (
```

```
    'A' = GoBack,
    'B' = RefreshData,
    'C' = SalesChart
  )
);
```

ALTER report objects

`ALTER` modifies one or more properties of an existing report object without redefining it. Any clause omitted keeps its current value.

```
-- Change a visual's title and source
ALTER VISUAL RevenueByRegion (
  TITLE  = 'Revenue by Region (Updated)',
  SOURCE = #new_summary
);

-- Add a tooltip to an existing page
ALTER PAGE Overview (
  TOOLTIP = 'Live sales data, refreshed hourly'
);

-- Update container visuals list
ALTER CONTAINER KpiRow (
  VISUALS (TotalRevenue, TotalUnits, AvgOrderValue)
);

-- Change a button's label
ALTER BUTTON GoBack (
  TITLE = '← Return'
);

-- Rename a style property
ALTER STYLE DarkCard (
  BACKGROUND-COLOR = '#2a2a3e'
);
```

Supported object types: `VISUAL`, `PAGE`, `CONTAINER`, `BUTTON`, `STYLE`, `NAVIGATION`, `DATASET`

CREATE OR ALTER

`CREATE OR ALTER` is equivalent to `ALTER` if the object already exists, or `CREATE` if it does not. This is useful for idempotent scripts that are run repeatedly.

```
CREATE OR ALTER VISUAL TotalRevenue AS CARD (
  SOURCE   = (SELECT SUM(revenue) AS val FROM #summary),
  TITLE    = 'Total Revenue',
```

```
  MAPPINGS (VALUE = val)
);

CREATE OR ALTER STYLE DarkCard (
  BACKGROUND-COLOR = '#1e1e2e',
  COLOR            = '#cdd6f4',
  BORDER-RADIUS    = '8px'
);
```

Supported for all report object types: `VISUAL`, `PAGE`, `CONTAINER`, `BUTTON`, `STYLE`, `NAVIGATION`, `DATASET`.

DROP report objects

Permanently removes a report object from the execution context.

```
DROP VISUAL [IF EXISTS] <name>;
DROP PAGE [IF EXISTS] <name>;
DROP CONTAINER [IF EXISTS] <name>;
DROP BUTTON [IF EXISTS] <name>;
DROP STYLE [IF EXISTS] <name>;
DROP NAVIGATION [IF EXISTS] <name>;
DROP DATASET [IF EXISTS] <name>;
```

`IF EXISTS` suppresses the error when the named object does not exist. Without it, dropping a non-existent object raises an `ExecutionException`.

```
-- Clean up before rebuilding
DROP VISUAL IF EXISTS TotalRevenue;
DROP PAGE   IF EXISTS Overview;

-- Error if MyNav does not exist
DROP NAVIGATION MyNav;
```

CLI — etl-sql-report

build

Evaluates the script, builds a `ReportManifest`, and writes output files:

```
etl-sql-report build report.rptsql
etl-sql-report build report.rptsql --output out/dashboard.md
etl-sql-report build report.rptsql --format json
etl-sql-report build report.rptsql --format pdf
```

Output files produced:

File	Description
`<script>.report.md`	GitHub Flavored Markdown document. Default when `--format md`.
`<script>.report.json`	Raw manifest JSON. Default when `--format json`.
`<script>.report.pdf`	PDF export via QuestPDF. Charts rendered as SVGs, tables capped at 500 rows. Default when `--format pdf`.
`<script>.snapshot.json`	Snapshot of all visual data rows and metadata. Always written alongside the report.

Flags:

Flag	Default	Description
`--output`, `-o`	`<script>.report.<ext>`	Override the output file path.
`--format`, `-f`	`md`	Output format: `md`, `json`, or `pdf`.

refresh

Re-evaluates the script and updates the snapshot without writing a new report document:

```
etl-sql-report refresh report.rptsql
```

The snapshot is stored alongside the script as `<script>.snapshot.json`. The ReportPlayer considers the snapshot stale if the script file has been modified since the snapshot was built, or if the TTL (default 24 h) has elapsed.

serve

Starts the web dashboard at `http://localhost:5200`:

```
# Single report
etl-sql-report serve report.rptsql

# Multi-report catalog (see reports.json below)
etl-sql-report serve --manifest reports.json
```

Internally this launches `ETL-SQL.ReportPlayer` (the Kestrel ASP.NET server) and opens the browser after 2.5 s. Keep the process running; the dashboard is served for as long as the process is alive.

Multi-report hosting

Multiple reports can be hosted together using a `reports.json` manifest file:

```
{
  "reports": [
    { "name": "sales",     "path": "reports/sales.rptsql",     "description":
"Regional sales dashboard" },
    { "name": "inventory", "path": "reports/inventory.rptsql", "description":
"Inventory levels by SKU" }
  ]
}
```

Start the server:

```
etl-sql-report serve --manifest reports.json
```

The catalog page at `http://localhost:5200` lists all reports. Each report is accessible at `http://localhost:5200/reports/<name>`. API routes are prefixed per-report: `/reports/<name>/api/manifest`, `/reports/<name>/api/refresh`, etc.

ReportPlayer — web dashboard

The ReportPlayer is a lightweight ASP.NET Minimal API server that hosts the report as an interactive dashboard.

Endpoints (single-report mode)

Method	Path	Description
`GET`	`/`	Serves the full dashboard HTML with pre-embedded manifest.
`GET`	`/api/manifest`	Returns the current `ReportManifest` as JSON.
`GET`	`/api/refresh`	Triggers a full rebuild and returns `{ rebuilt: true, visuals: N }`.
`POST`	`/api/parameter`	Updates one parameter and triggers a selective rebuild. Body: `{ "name": "@region", "value": "West" }`.
`POST`	`/api/parameters`	Updates multiple parameters in a single request. Body: `{ "params": [{ "name": "@region", "value": "West" }, ...] }`.

Endpoints (multi-report mode)

Method	Path	Description
`GET`	`/`	Catalog page listing all reports.
`GET`	`/reports/{name}`	Dashboard for the named report.
`GET`	`/reports/{name}/api/manifest`	Report manifest JSON.

Method	Path	Description
GET	/reports/{name}/api/refresh	Force rebuild.
POST	/reports/{name}/api/parameter	Set one parameter.
POST	/reports/{name}/api/parameters	Set multiple parameters.

Selective refresh

When a parameter changes, DashboardService checks which visuals depend on that parameter (by scanning the inline SELECT for variable references) and re-queries only those visuals. Unaffected visuals keep their current data without re-evaluation.

Staleness banner

If the manifest was built before the script file was last written, or if more than 24 hours have passed, a yellow banner appears:

> ⚠ Snapshot may be stale — run etl-sql-report refresh to update.

You can also hit /api/refresh to force a live rebuild without restarting the server.

Dashboard rendering

There are two separate frontends depending on context:

- **VS Code WebviewPanel**: loads the bundled React application (ui/dist/index.html). The extension injects the manifest as window.__INITIAL_STATE__.messages and sets window.VIEW_TYPE = 'report'. The React app reads the initial manifest from that variable and receives subsequent refreshes via webview.postMessage() without re-mounting.
- **Web (ReportPlayer)**: a single vanilla-JS file (wwwroot/report-runtime.js). Uses the pre-embedded manifest in single-report mode or fetches from /api/manifest in multi-report mode.

Both frontends use Apache ECharts v5 for chart rendering.

VS Code preview

With a .rptsql file open, run **ETL-SQL: Preview Report** from the command palette or click the $(graph) icon in the editor title bar. A webview panel opens beside the editor and auto-refreshes every time you save the file.

Configuration:

Setting	Default	Description
etlsql.report.executable.path	etl-sql-report.exe	Full path to etl-sql-report.exe. Leave empty to use dotnet run from the source tree in development.

Setting	Default	Description
`etlsql.report.autoOpenPreview`	`false`	Automatically open the Report Preview panel when opening an `.rptsql` file.

Linter rules (language server)

The language server checks `.rptsql` files automatically:

Rule	Severity	Condition
`VisualSourceExists`	Warning	`SOURCE = #table` references a temp table not defined earlier in the script.
`VisualMappingColumnExists`	Warning	A `MAPPINGS` role references a column not returned by the `SOURCE` inline SELECT.
`PageVisualReferenced`	Warning	A `MAP` entry references a visual or container name that is not defined in the script.
`DatasetRefreshInterval`	Warning	`REFRESH EVERY` value does not match the `<n>s/m/h/d` format.
`DatasetEncryptWithoutKey`	**Error**	`ENCRYPT = KEYFILE` without a `KEYFILE` clause, or `ENCRYPT = PASSWORD` without a `PASSWORD` clause.
`LayerOrder`	Warning	A visual references a dataset defined later in the script, or a page references a visual defined later. Forward references are not supported.
`InsertColumnCountMismatch`	Warning	`INSERT INTO` omits the column list and the SELECT provides fewer columns than the target table.

ReportManifest JSON schema

The snapshot and API both return this structure:

```
{
  "source":      "C:/reports/sales.rptsql",
  "builtAt":     "2026-04-12T18:00:00Z",
  "title":       "Sales Dashboard",
  "description": "Regional revenue analysis",

  "visuals": [
    {
      "name":       "RevenueByRegion",
      "visualType": "Bar",
      "chartConfig": "{ /* ECharts option object JSON */ }",
      "columns":  ["region", "revenue"],
      "rows":     [["East", "12000"], ...],
      "options":  {
```

```
        "mapping:x":       "region",
        "mapping:y":       "revenue",
        "axis:x:label":    "Region",
        "axis:y:label":    "Revenue ($)"
      },
      "styles":   { "THEME": "dark" },
      "actions":  [
        { "type": "SET_PARAMETER", "trigger": "ON_CHANGE",
          "parameterName": "@region", "valueExpression": "region" }
      ]
    }
  ],

  "pages": [
    {
      "name":      "Overview",
      "structure": "A B / C C",
      "slotMap":   { "A": "TotalRevenue", "B": "RegionFilter", "C":
"RevenueByRegion" },
      "parameters": { "@region": "All" },
      "styles":    { "THEME": "dark" }
    }
  ],

  "containers": [
    {
      "name":          "KpiRow",
      "containerType": "BOX",
      "visuals":       ["TotalRevenue", "TotalUnits"],
      "styles":        { "HEIGHT": "200" }
    }
  ],

  "navigations": [
    {
      "name":        "MainNav",
      "navType":     "TAB",
      "orientation": "HORIZONTAL",
      "defaultPage": "Overview",
      "pages":       ["Overview", "Details"]
    }
  ],

  "datasets": [
    {
      "tempTableName":   "#sales_snap",
      "refreshInterval": "1h",
      "ttl":             "24h",
      "lastRefresh":     "2026-04-12T18:00:00Z",
      "rowCount":        4800
    }
  ]
}
```

Full working example

Source CSV columns: `region`, `product`, `units`, `revenue`, `month`

```
SET REPORT TITLE = 'Sales Dashboard';
SET REPORT DESCRIPTION = 'Regional and product-level revenue by month.';

DROP CONNECTION IF EXISTS c;
CREATE CONNECTION c ON FLATFILE('TestData/test_sales.csv');

-- Shared base dataset (refreshed every hour)
CREATE DATASET #summary
  REFRESH EVERY '1h'
  ENCRYPT = MACHINE
  AS (SELECT month, region, product,
             SUM(units)   AS units,
             SUM(revenue) AS revenue
      FROM c
      GROUP BY month, region, product);

-- Region filter slicer
CREATE VISUAL RegionFilter AS SLICER (
  SOURCE   = (SELECT DISTINCT region FROM #summary ORDER BY region),
  MAPPINGS (VALUE = region),
  ACTIONS  (ON_CHANGE = SET_PARAMETER(@region, region)),
  DEFAULT  = 'All'
);

-- KPI card: total revenue
CREATE VISUAL TotalRevenue AS CARD (
  SOURCE = (SELECT SUM(revenue) AS val, 'Total Revenue' AS lbl
            FROM #summary
            WHERE @region = 'All' OR region = @region),
  TITLE    = 'Total Revenue',
  MAPPINGS (VALUE = val, LABEL = lbl),
  OPTIONS  (FORMAT = 'C0')
);

-- Bar chart: revenue by region
CREATE VISUAL RevenueByRegion AS BAR (
  SOURCE = (SELECT region, SUM(revenue) AS revenue
            FROM #summary
            WHERE @region = 'All' OR region = @region
            GROUP BY region),
  TITLE    = 'Revenue by Region',
  MAPPINGS (X = region, Y = revenue),
  OPTIONS (
    X_AXIS (LABEL = 'Region'),
    Y_AXIS (LABEL = 'Revenue ($)', MIN = 0)
  )
```

```
);

-- Multi-series bar: revenue by region and month
CREATE VISUAL RevenueByRegionMonth AS BAR (
  SOURCE = (SELECT month, region, SUM(revenue) AS revenue
            FROM #summary
            GROUP BY month, region),
  TITLE    = 'Revenue by Region (Monthly)',
  MAPPINGS (X = month, Y = revenue, SERIES = region),
  OPTIONS  (STACKED = ON, X_AXIS (LABEL = 'Month'), Y_AXIS (LABEL = 'Revenue ($)',
MIN = 0))
);

-- Donut chart: revenue share by product
CREATE VISUAL RevenueByProduct AS DONUT (
  SOURCE = (SELECT product, SUM(revenue) AS revenue
            FROM #summary
            GROUP BY product),
  TITLE    = 'Revenue by Product',
  MAPPINGS (LABEL = product, VALUE = revenue)
);

-- Line chart: units by month
CREATE VISUAL UnitsByMonth AS LINE (
  SOURCE = (SELECT month, SUM(units) AS units
            FROM #summary
            GROUP BY month),
  TITLE    = 'Units Sold by Month',
  MAPPINGS (X = month, Y = units),
  OPTIONS  (SMOOTH = ON, X_AXIS (LABEL = 'Month'), Y_AXIS (LABEL = 'Units Sold',
MIN = 0))
);

-- Detail table: all rows
CREATE VISUAL SalesTable AS TABLE (
  SOURCE = #summary
);

-- KPI container
CREATE CONTAINER KpiRow AS BOX (
  VISUALS (TotalRevenue)
);

-- Dashboard pages
CREATE PAGE Overview AS LAYOUT (
  STRUCTURE = 'A B / C C / D D',
  MAP (
    'A' = KpiRow,
    'B' = RegionFilter,
    'C' = RevenueByRegion,
    'D' = SalesTable
  )
)
WITH PARAMETERS (@region = 'All');
```

```
CREATE PAGE Trends AS LAYOUT (
  STRUCTURE = 'A A / B C',
  MAP (
    'A' = RevenueByRegionMonth,
    'B' = UnitsByMonth,
    'C' = RevenueByProduct
  )
);

-- Navigation (defined after pages so the LayerOrder linter is satisfied)
CREATE NAVIGATION MainNav AS TAB (
  ORIENTATION = HORIZONTAL,
  DEFAULT = Overview
)
WITH PAGES (Overview, Trends);
```
